

Bootcamp: Full Stack Junior

Modulo 3: Backend con PHP y Laravel

Semana 7

Contenido

1

Patrón Modelo-Vista-Controlador (MVC).

2

Ciclo de vida de una petición en Laravel.

3

Estructura de directorios y Service Providers.

Contenido:

1. Introducción al Backend con Laravel

Laravel es un framework de PHP para desarrollar aplicaciones web modernas siguiendo el patrón MVC (Modelo–Vista–Controlador), lo que ayuda a organizar el código y hacerlo más mantenible. Se usa para crear desde APIs hasta aplicaciones complejas y escalables, con una sintaxis clara y muchas herramientas ya integradas.



Ilustración 1 - Laravel - Fuente: Laravel

1.1. ¿Qué es exactamente Laravel?

- Es un framework de código abierto basado en PHP, con una arquitectura MVC que separa lógica de negocio, vistas y controladores.
- Incluye un ORM llamado Eloquent para trabajar con bases de datos usando objetos en lugar de escribir SQL crudo.
- Trae herramientas listas para uso común: enrutamiento, migraciones, autenticación, colas, caché, entre otras.

1.2. ¿Por qué usar Laravel?

- Sintaxis elegante y estructura organizada que hace el código más legible y fácil de mantener, reduciendo errores y tiempo de desarrollo.
- Sistema de rutas simple y expresivo que facilita definir URLs limpias y manejarlas en controladores.

- Sistema de autenticación integrado (login, registro, recuperación de contraseña) con protección frente a XSS e inyección SQL.
- Buen rendimiento gracias al soporte de caché integrado y mecanismos para manejar tráfico alto.
- Muy buena escalabilidad y mantenimiento a largo plazo gracias a su arquitectura modular y diseño orientado a objetos.
- Ecosistema maduro: herramientas como artisan (CLI), integración con Composer, y muchos paquetes de la comunidad que aceleran el desarrollo.

1.3. ¿En qué tipo de proyectos se usa?

- Aplicaciones web a medida (paneles de administración, sistemas internos de empresas, CRMs, ERPs, etc.).
- APIs para aplicaciones móviles o SPAs hechas con frameworks frontend (React, Vue, etc.).
- E-commerce, sitios de contenido, redes sociales y aplicaciones empresariales de varios niveles.

1.4. ¿En qué tipo de proyectos se usa?

Empresa/Proyecto	Descripción	Beneficios
CentoSuit	Automatización financiera interna con reporting real-time.	Redujo trabajo manual 50%, centralizó datos.
Wayfinder	Paneles admin para inventarios/empleados con Inertia.js/Vue.	SPA sin API separada, rápido para apps internas.
Ace Infoway/Perfsol	Backoffice en fintech/salud/hospitalidad (SaaS, DevOps).	+1000 proyectos, retención 93%.

Tabla 1 - Casos de uso de Laravel - Fuente: Autor de Contenidos

El framework Laravel tiene un gran ecosistema para la construcción de aplicaciones tanto pequeñas como robustas y todo su ecosistema se compone de la siguiente manera:

Ecosistema Laravel

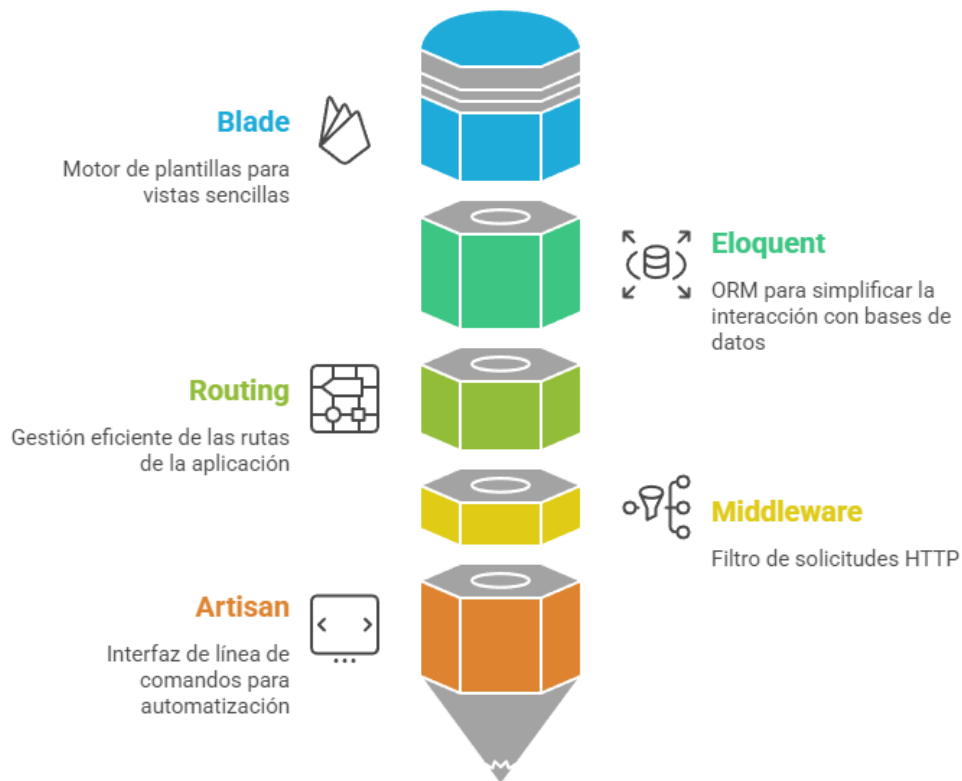


Ilustración 2- Ecosistema de Laravel - Fuente: Autor de contenidos

2. Patrón Modelo–Vista–Controlador (MVC)

El patrón MVC organiza el código separando datos, interfaz y lógica. En Laravel, se implementa de forma natural para apps web escalables.

Flujo básico en Laravel

Usuario → Controlador (app/Http/Controllers) → Modelo (app/Models) → Base de datos → Vista (resources/views) → Usuario.

Tabla 2 - Patrón MVC en Laravel- Fuente: Autor de Contenidos

2.1. Componentes clave de MVC en Laravel

- **Modelo**: Maneja datos y DB con Eloquent ORM (ej: `User::find(1)`). Carpeta: `app/Models`.
- **Vista**: Plantillas Blade (`.blade.php`) que muestran HTML con datos (ej: `@foreach($users as $user)`). Carpeta: `resources/views`.

- **Controlador:** Recibe requests HTTP, llama Modelo, pasa datos a Vista (ej: `return view('users', ['users' => User::all()])`). Carpeta: `app/Http/Controllers`.

Ejemplo visual de cómo funciona Laravel.



Ilustración 3- Funcionamiento de Laravel - Fuente: GPT Image

3. Estructura de directorios y Service Providers.

Laravel por defecto ofrece un gran ecosistema de herramientas listas para ser integradas a funcionalidades comunes que todo backend tarde o temprano necesita. Las herramientas que Laravel por defecto ya ofrece son las siguientes:

Authentication: Sistema completo para registro, login, logout y recuperación de contraseñas. Maneja la seguridad de usuarios automáticamente.

Authorization: Controla quién puede hacer qué en tu aplicación mediante políticas y gates. Define permisos y roles fácilmente.

Eloquent ORM: Trabaja con bases de datos usando objetos PHP en lugar de SQL directo. Cada tabla se convierte en un modelo elegante y expresivo.

Database migrations: Versionamiento de tu base de datos mediante código. Crea, modifica o revierte cambios en tablas como si fueran commits de Git.

Validation: Valida datos de formularios y APIs con reglas simples. Mensajes de error automáticos en múltiples idiomas.

Notification and mail: Envía notificaciones por email, SMS, Slack o base de datos desde un solo lugar con sintaxis unificada.

File storage: Maneja archivos locales, Amazon S3 o cualquier sistema de almacenamiento con la misma API simple.

Job queues: Procesa tareas pesadas en segundo plano para que tu aplicación responda rápido. Soporta múltiples servicios de cola.

Task scheduling: Programa tareas automáticas (cada hora, diarias, semanales) desde código PHP sin configurar cron manualmente.

Testing: Herramientas integradas para escribir tests unitarios y de funcionalidad. Simula peticiones HTTP y verifica respuestas fácilmente.

Events and WebSockets: Sistema de eventos para desacoplar tu código. Integración con WebSockets para funcionalidades en tiempo real como chats.

Para crear un proyecto Laravel es necesario instalar el gestor de paquetes de php llamado *Composer* y posteriormente, Ejecuta en la terminal de línea de comandos lo siguiente:



```
1 composer global require laravel/installer
```

Ilustración 4- Instalador de Laravel - Fuente: Laravel

Este comando instalara toda la paquetería necesaria para poder crear proyectos de Laravel. El siguiente comando para ejecutar es:



```
1 laravel new example-app
```

Ilustración 5- Comando de creación de App de Laravel - Fuente: Laravel

Una vez creada la aplicación, puedes iniciar el servidor de desarrollo local de Laravel, ejecutando los siguientes comandos:



```
1 cd example-app
2 npm install && npm run build
3 composer run dev
```

Ilustración 6- Comandos dev server de Laravel - Fuente: Laravel

Una vez iniciado el servidor de desarrollo, su aplicación estará disponible en su navegador web en <http://localhost:8000>.

3.1. Estructura de proyecto de Laravel

La estructura predeterminada de la aplicación Laravel está diseñada para proporcionar un excelente punto de partida tanto para aplicaciones grandes como pequeñas. Sin embargo, puedes organizar tu aplicación como prefieras. Laravel prácticamente no impone restricciones sobre la ubicación de una clase, siempre que Composer pueda cargarla automáticamente.

(Manual de estructura de proyectos adjunto en pdf)

3.2. Artisan

Es la interfaz de línea de comandos de Laravel; te permite generar código, manejar la base de datos, limpiar cachés y ejecutar tareas internas del framework con comandos

- Es el CLI (Command Line Interface) oficial que viene en cualquier proyecto Laravel, ubicado en el archivo artisan en la raíz del proyecto.

- Proporciona decenas de comandos para automatizar tareas típicas: generar clases, correr migraciones, limpiar caché, manejar colas, etc.
- También permite crear tus propios comandos personalizados para automatizar procesos específicos de tu aplicación.

Ejemplo: en la raíz de tu proyecto puedes ejecutar `php artisan list` para ver todos los comandos disponibles.

3.3. ¿Para qué se usa Artisan en Laravel?

Usos principales:

- Generación de código
 - Crear modelos, controladores, migraciones, factories, seeders, form requests, jobs, eventos, notificaciones, etc., con comandos `make:*`.
- Gestión de base de datos
 - Ejecutar migraciones (`migrate`), revertir cambios (`migrate:rollback`), sembrar datos (`db:seed`), refrescar la base (`migrate:fresh`).
- Caché, rutas y configuración
 - Limpiar y generar cachés de config, rutas y vistas con `config:cache`, `route:cache`, `view:cache`, `cache:clear`.
- Servidor de desarrollo
 - Levantar el servidor local con `serve` (útil en desarrollo rápido).
- Tareas en segundo plano
 - Manejar colas (`queue:work`, `queue:listen`), trabajos diferidos, y tareas programadas vía `schedule:run` en el cron.
- Depuración y entorno interactivo
 - Usar `tinker` para ejecutar código PHP/Laravel en una consola interactiva, probar Eloquent, helpers, etc.

Todo esto aumenta la productividad, mantiene consistencia en la estructura del proyecto y reduce errores manuales.

4. Ciclo de vida de una petición en Laravel.

El ciclo de vida de una petición en Laravel describe cómo se procesa una solicitud HTTP desde su entrada hasta la respuesta final. Comienza en `public/index.php`, que carga el autoloader de Composer y crea la aplicación desde `bootstrap/app.php`.

4.1. Flujo completo paso a paso

1. Entrada: `public/index.php` captura la request y la pasa al Kernel HTTP (`app/Http/Kernel.php`).
2. Service Providers: Se registran e inicializan componentes (DB, cache, auth) desde `config/app.php`.
3. Middleware global: Filtros como web o api (CSRF, cookies, rate limiting).
4. Router: Matching en `routes/web.php` o `api.php` → Controller o closure.
5. Middleware de ruta: Filtros específicos (`auth:sanctum`, `role:admin`).
6. Controller/Modelo: Lógica negocio + consulta DB con Eloquent → datos procesados.
7. Vista/Response: Renderiza Blade o JSON → Response object.
8. Middleware inverso: Response pasa por filtros de salida.
9. Terminate: `Kernel::terminate()` ejecuta tareas finales (logs, queues).
10. Send: `$response->send()` entrega HTML/JSON al browser.

4.2. Diagrama del ciclo

Ciclo de Vida de una Petición en Laravel



Ilustración 7- Ciclo de vida de una petición en Laravel - Fuente: GPT Image

Laravel es un framework de PHP moderno y elegante que simplifica el desarrollo web mediante una sintaxis expresiva y herramientas integradas. Ofrece un ecosistema completo con sistema de enrutamiento, ORM (Eloquent), autenticación, manejo de sesiones, colas, y mucho más, todo listo para usar.

Laravel destaca por su curva de aprendizaje amigable comparado con otros frameworks, pero sin comprometer funcionalidades avanzadas. Además, cuenta con herramientas como Laravel Forge para deployment, Laravel Vapor para serverless, y un ecosistema de paquetes (Livewire, Breeze, Jetstream) que extienden sus capacidades.

Recursos de Apoyo



Material de Lectura

[Documentación oficial: Guía completa con instalación, MVC, rutas, Eloquent y novedades](#)



Guia

[Guía: Instalación, server dev, basics en Laravel 12.](#)



Video

[Tutorial: Como iniciar con Laravel.](#)